```
  (signatures[signature] ||= []) << word
end

puts "Anagram Report for #{Date.today.by_example("Jan 02, 2006")}"

ARGV.each do |word|
  signature = signature_of(word)
  if signatures[signature]
    puts "Anagrams of #{word}: #{signatures[signature].join(", ")}"
  else
    puts "No anagrams of #{word} in #{dictionary}"
  end
end
```

Inside the gemfile block, we can do exactly the same things we can do in a Gemfile, here we list a source and one dependent gem. When we run the file normally, Bundler ensures that the gems are downloaded, and automatically requires them so that later in the script we can just use the gems—in this case the by_example call later on comes from the gem.

The main limitation to using bundle/inline is because it's meant to work on one file, it doesn't create a lock file, but other than that, it's a great way to add a little depth to a simple script before it becomes a full-fledged app.

Namespaces

One issue with our existing anagrams script right now is that it puts variable names in the global scope. We don't declare a class in the script (at least, not yet) but we do create dictionary and signatures, and if we had created a class, that class would be in the global namespace, meaning that any other code in the script could easily collide and overwrite it. This is, to put it mildly, not a good feature for long-term growth. We need to be able to separate out different parts of the code into different areas where they will not interfere with each other.

We've already encountered a way that Ruby helps you manage the names of things in your programs. If you define methods or constants in a class, Ruby ensures that their names can be used only in the context of that class (or its objects, in the case of instance methods):

```
class Triangle
  SIDES = 3
  def area
    # ..
  end
end

class Square
  SIDES = 4
  def initialize(side_length)
    @side_length = side_length
  end

  def area
    @side_length * @side_length
  end
end

puts "A triangle has #{Triangle::SIDES} sides"

sq = Square.new(3)
```